UNIT 3

Protocol Standardization for IoT

Protocol standardization in IoT ensures consistent communication, interoperability, and security across diverse devices and systems. Below is a structured explanation of its key aspects:

1. What is IoT Protocol Standardization?

IoT protocols are predefined rules governing how devices exchange data. Standardization involves creating universal specifications for these protocols to ensure compatibility across manufacturers and industries. This prevents fragmentation, reduces costs, and enhances security.

Example:

- **MQTT** (Message Queuing Telemetry Transport) is a standardized pub/sub protocol optimized for low-bandwidth, high-latency environments.
- **Zigbee** follows IEEE 802.15.4 standards for low-power mesh networks in smart homes.

2. Key Standardized IoT Protocols

Protocol	Function	Layer	Use Case
CoAP	HTTP-like communication for constrained devices	Application	Smart meters, industrial sensors
6LoWPAN	Enables IPv6 over low-power networks	Network	Home automation, healthcare
LoRaWAN	Long-range, low-power wide-area networks	Physical/Data Link	Agriculture, smart cities
OPC UA	Secure industrial data exchange	Application	Manufacturing, IIoT
Matter	Unified smart home interoperability	Application	Consumer IoT devices

3. Organizations Driving Standardization

- **IEEE**: Develops foundational standards like IEEE 802.15.4 for Zigbee and 6LoWPAN.
- **IETF**: Defines protocols such as CoAP and 6LoWPAN.
- **ETSI**: Oversees cellular IoT standards (e.g., LTE-M, NB-IoT) and collaborates with **oneM2M** for service-layer interoperability.
- **OMA SpecWorks**: Manages **LwM2M** for device management in low-power environments.

4. Benefits of Standardization

- **Interoperability**: Devices from different vendors can communicate seamlessly (e.g., Matter unifying smart home ecosystems).
- **Security**: Standards like **DTLS** (Datagram Transport Layer Security) encrypt data in CoAP, while **Ascon** (NIST-approved) secures IoT encryption.
- **Scalability**: LPWAN protocols (e.g., LoRaWAN) support thousands of devices over wide areas.
- **Cost Efficiency**: Reduced development time and compatibility testing.

5. Challenges in IoT Standardization

- **Fragmentation**: Competing protocols (e.g., Zigbee vs. Z-Wave) create market silos.
- **Power Constraints**: Protocols must balance energy efficiency with functionality (e.g., BLE vs. Wi-Fi).
- **Legacy Systems**: Integrating older devices with new standards (e.g., retrofitting 3G devices for LTE-M).
- **Security Gaps**: Inconsistent encryption methods across protocols.

just to be clear: silos example: The Android and IOS platforms create market silos because they offer distinct ecosystems with unique app stores, hardware and user experiences. This fragmentation impacts app developers, users and overall market dynamics.

6. Future Trends

- **5G Integration**: Enabling ultra-reliable, low-latency communication for autonomous systems [10].
- **Semantic Interoperability**: Projects like **SAREF** (Smart Applications REFerence ontology) standardize data meaning across domains [8].
- **Edge Computing**: Protocols like **DDS** (Data Distribution Service) optimize real-time data processing [710].

Things to remember:

Protocol standardization is critical for IoT's growth, ensuring devices communicate reliably and securely. While challenges like fragmentation persist, initiatives like Matter and 5G are paving the way for unified ecosystems.

IoT Protocol Standardization Efforts

Current status of IoT Standardization:

- Fragmented architectures, no coherent unifying conception, solution existing only for application silos.
- No holistic approach to implement the IoT system has yet been approved.
- Little cross-sector reuse of technology and exchange of knowledge.

Efforts Being Made for Standardization in IoT

To solve the issues caused by the lack of standardization in IoT, many global organizations and industries are actively working on setting common standards. These efforts focus on **communication, security, interoperability**, and **data management**.

1. Development of Communication Protocols

- **Organizations Involved**:
  - **IEEE** (Institute of Electrical and Electronics Engineers)
  - **IETF** (Internet Engineering Task Force)
  - **ITU** (International Telecommunication Union)
- **Examples of Standards**:
  - **IEEE 802.15.4**: Standard for low-rate wireless personal area networks (used in Zigbee).
  - **IPv6 over Low-Power Wireless Personal Area Networks (6LoWPAN)**: Allows IoT devices to use internet protocols efficiently.

2. Creation of IoT-Specific Platforms and Frameworks

- **oneM2M**: A global initiative that creates a common machine-to-machine (M2M) service layer that can be embedded in hardware and software.
- **OMA LwM2M (Open Mobile Alliance Lightweight M2M)**: A device management protocol for low-power devices.

3. Security and Privacy Standards

- **ISO/IEC 27001**: Information security management.
- **NIST Cybersecurity Framework (USA)**: Guidelines to secure IoT systems.
- **IoTSF (IoT Security Foundation)**: Promotes best practices for securing devices.

4. Industrial Alliances and Consortiums

These groups bring companies together to collaborate on standards:

- **Industrial Internet Consortium (IIC)**: Works on standardizing industrial IoT.
- **AllSeen Alliance / AllJoyn**: Promotes interoperability across devices.
- **Open Connectivity Foundation (OCF)**: Develops open standards for device-to-device communication.
- **Thread Group**: Focuses on mesh networking for smart homes.

5. Government and Regulatory Involvement

- **European Union**: Introduced regulations like GDPR to ensure secure and private IoT deployments.
- **US FCC and NIST**: Create and enforce communication and cybersecurity standards.
- **India**: Published "IoT Policy" with guidelines for security, data handling, and spectrum management.

6. Use of Open Source and Interoperability Initiatives

- **Eclipse IoT**: Provides open-source tools and frameworks for building IoT applications.
- **FIWARE**: Open platform for smart solutions with standard APIs and data models.

little bubble

Issues with Standardization in IoT

Standardization in IoT is crucial to ensure interoperability, security, and scalability across various devices and platforms. However, due to the complexity and rapid growth of IoT technologies, several issues arise:

1. Lack of Universal Standards

- **Problem**: Different manufacturers use their own communication protocols and data formats.
- **Impact**: Devices from different vendors may not work together, making integration difficult.
- **Example**: A smart bulb from one company may not communicate with a smart home hub from another.

2. Fragmentation Across Layers

IoT systems include multiple layers: **device, network, data, and application**.

- **Problem**: Each layer may use different standards or none at all.

- **Impact**: This creates compatibility issues between sensors, communication protocols (like Zigbee, Bluetooth, Wi-Fi), and platforms.
- **Example**: An industrial sensor using MQTT may not easily integrate with a platform using CoAP.

3. Security and Privacy Standards

- **Problem**: There are no consistent global standards for securing IoT devices and user data.
- **Impact**: Devices may be vulnerable to hacking, data theft, and surveillance.
- **Example**: Weak encryption or default passwords can be exploited, compromising networks.

4. Scalability and Network Compatibility

- **Problem**: Different communication protocols (LoRaWAN, NB-IoT, Zigbee, etc.) are designed for different use cases.
- **Impact**: Difficult to scale systems or switch networks without compatibility problems.
- **Example**: A device built for Wi-Fi may not work in areas where LoRaWAN is more suitable.

5. Legal and Regulatory Differences

- **Problem**: Countries have different laws and regulations related to frequency usage, data storage, and communication.
- **Impact**: A product that works in one country may be illegal or non-functional in another.
- **Example**: Radio frequency standards vary between the US and Europe.

6. Rapid Technological Changes

- **Problem**: IoT technology evolves faster than standardization processes.
- **Impact**: New technologies emerge without established standards, leading to temporary chaos.
- **Example**: Emerging 5G IoT applications still lack unified standards for certain use cases.

Things to remember:

Standardization is essential for building reliable and secure IoT systems. However, the current landscape is fragmented due to various factors such as lack of universal protocols, security inconsistencies, and differing regulations. Solving these issues requires collaboration between governments, industries, and standardization bodies.

M2M Communication (Machine-to-Machine)

M2M refers to automated, direct communication between devices over wired or wireless networks **without human intervention**. It forms the backbone of IoT ecosystems, enabling devices to exchange data, trigger actions, and optimize processes. Below is a structured overview:

1. Key Components of M2M

1. **Devices/Sensors**:
    - Collect data (e.g., temperature sensors, GPS trackers).
2. **Communication Protocols**:
    - **MQTT**, **CoAP**, **HTTP** for lightweight data transfer.
    - **LPWAN** protocols (e.g., **NB-IoT**, **LoRaWAN**) for low-power, long-range communication.
3. **Networks**:
    - Cellular (4G/5G), Wi-Fi, satellite, or wired connections.
4. **Central Management System**:
    - Processes data, issues commands, and monitors device health (e.g., cloud platforms like AWS IoT).

2. How M2M Works

- **Step 1**: Device A (e.g., a smart meter) collects data.
- **Step 2**: Data is transmitted via a network to a central server.
- **Step 3**: Server processes data and sends commands to Device B (e.g., shutting off a valve).
- **Step 4**: Actions are executed autonomously.

3. Applications of M2M

Industry	Use Case
Healthcare	Remote patient monitoring (e.g., pacemakers).
Automotive	Telematics for fleet tracking and diagnostics.
Utilities	Smart grids for real-time energy management.
Retail	Automated inventory tracking and restocking.

Industry	Use Case
Agriculture	Precision farming via soil/weather sensors.

4. M2M vs. IoT

Aspect	M2M	IoT
Scope	Device-to-device communication.	Broader ecosystem (devices + cloud + analytics).
Data Use	Immediate actions (e.g., alerts).	Data aggregation for insights/trends.
Connectivity	Often uses point-to-point networks.	Relies on IP-based, cloud-integrated networks.

5. Standards & Protocols

- **oneM2M**: Global standard for interoperability across verticals (e.g., smart cities, healthcare).
- **3GPP**: Manages cellular M2M standards like **LTE-M** and **NB-IoT** for scalable deployments.
- **OMA-DM**: Protocol for remote device management (e.g., firmware updates).

Things to remember:

M2M is foundational to IoT, enabling seamless automation in industries like healthcare, utilities, and manufacturing. While challenges like security and interoperability persist, advancements in 5G, edge computing, and standardized protocols (e.g., oneM2M) are driving its evolution.

MQTT (Message Queuing Telemetry Transport)

Introduction:

- It's a **M2M** IoT connectivity protocol that establishes communication between devices.
- extremely lightweight and **publish-subscribe** messaging transport protocol.
- it doesn't require that both the client and server establish a connection at the same time.
- provides faster data transmission and faster delivery too than the system without mqtt.
- allows clients to subscribe the narrow selection of **topics** so that they can receive the information they are looking for.
- works over **TCP/IP**, commonly using port **1883** (or 8883 for secure connections
- designed for **low-power**, **low-bandwidth**, **high-latency**, or **unreliable** networks.

Architecture:

Key Components of MQTT Architecture

Component	Role
Broker	Central server that manages all message traffic
Client	Any device (sensor, microcontroller, etc.) that connects to the broker
Publisher	A client that sends messages to a topic
Subscriber	A client that receives messages from a topic
Topic	A string that labels each message channel (**like a folder path)** eg. home/temperature

How It Works (Step-by-Step)

1. **Connection Setup**
   - Clients (publishers/subscribers) connect to the MQTT **broker** using TCP/IP.
2. **Publish**
   - A **publisher** sends data to a **topic** (e.g., home/temperature).
   - The broker **does not store** the data unless it's a **retained message**.
3. **Subscribe**
   - A **subscriber** registers interest in a topic.
   - If a message is published on that topic, the **broker forwards** it to all subscribers.

4. **Broker Role**
    - The broker is the **core** of the system.
    - It filters messages, manages clients, and ensures delivery according to **QoS levels**.

Quality of Service (QoS)

Level	Description
0	At most once – No guarantee
1	At least once – May be duplicated
2	Exactly once – Ensures single delivery

WSN: Wireless Sensor Networks

A **Wireless Sensor Network (WSN)** is a collection of spatially distributed autonomous sensors that monitor physical or environmental conditions (e.g., temperature, sound, pressure, motion) and transmit data wirelessly to a central location.

1. Key Components of WSNs

- **Sensor Nodes**: Small, low-power devices with:
    - **Sensors** (to detect environmental data)
    - **Processor** (for data processing)
    - **Transceiver** (for wireless communication)
    - **Power source** (often batteries or energy harvesting)
- **Base Station (Sink Node)**: Collects data from sensor nodes and connects to external networks (e.g., the Internet).
- **Communication Infrastructure**: Wireless protocols (e.g., Zigbee, LoRa, Wi-Fi, Bluetooth) for data transmission.

2. Characteristics of WSNs

✔**Self-organizing**: Nodes form networks without manual configuration.

✔**Energy-constrained**: Many sensors rely on batteries or energy harvesting.

✔**Scalable**: Thousands of nodes can be deployed over large areas.

✔**Dynamic topology**: Nodes may fail or move, requiring adaptive routing.

✔**Limited processing & memory**: Optimized for low-power operation.

Bluetooth

Bluetooth is a **short-range wireless communication** technology used to connect devices like phones, headphones, laptops, microcontrollers, etc.

1. Bluetooth System Overview

Bluetooth consists of:

- **Master Device**: Controls the communication (e.g., smartphone).
- **Slave Device(s)**: Respond to the master's commands (e.g., Bluetooth speaker).

A **master can connect up to 7 active slave devices** in a network called a **Piconet**.

2. Bluetooth Network Topologies

Piconet

- One master + up to 7 active slaves.
- Temporary and automatically formed.
- Example: Phone (master) connected to a speaker and smartwatch (slaves).

Scatternet

- A group of interconnected piconets.
- A device can be part of multiple piconets (acting as a bridge).
- Example: Smartwatch communicating with both phone and laptop.

3. Bluetooth Versions (Brief)

Version	Speed	Features
1.0–2.0	~1 Mbps	Basic wireless communication

Version	Speed	Features
3.0	~24 Mbps	High-speed via Wi-Fi
4.0	~1 Mbps	Introduced **Bluetooth Low Energy (BLE)**
5.0+	~2 Mbps+	Improved range, speed, and mesh support

4. Bluetooth Devices (Examples)

- Phones
- Wireless headphones
- Smartwatches
- Wireless keyboards/mice
- ESP32 microcontrollers (with built-in Bluetooth)

SCADA

- Supervisory Control and Data Acquisition (SCADA) is a comprehensive hardware and software system that controls and manages high level industrial processes without human intervention.
- It's a type of industrial control system (ICS) used to monitor and control infrastructure and facility-based processes in industries like manufacturing, energy (power generation, oil & gas), water and waste water, transportation and smart buildings.

Key Components of SCADA

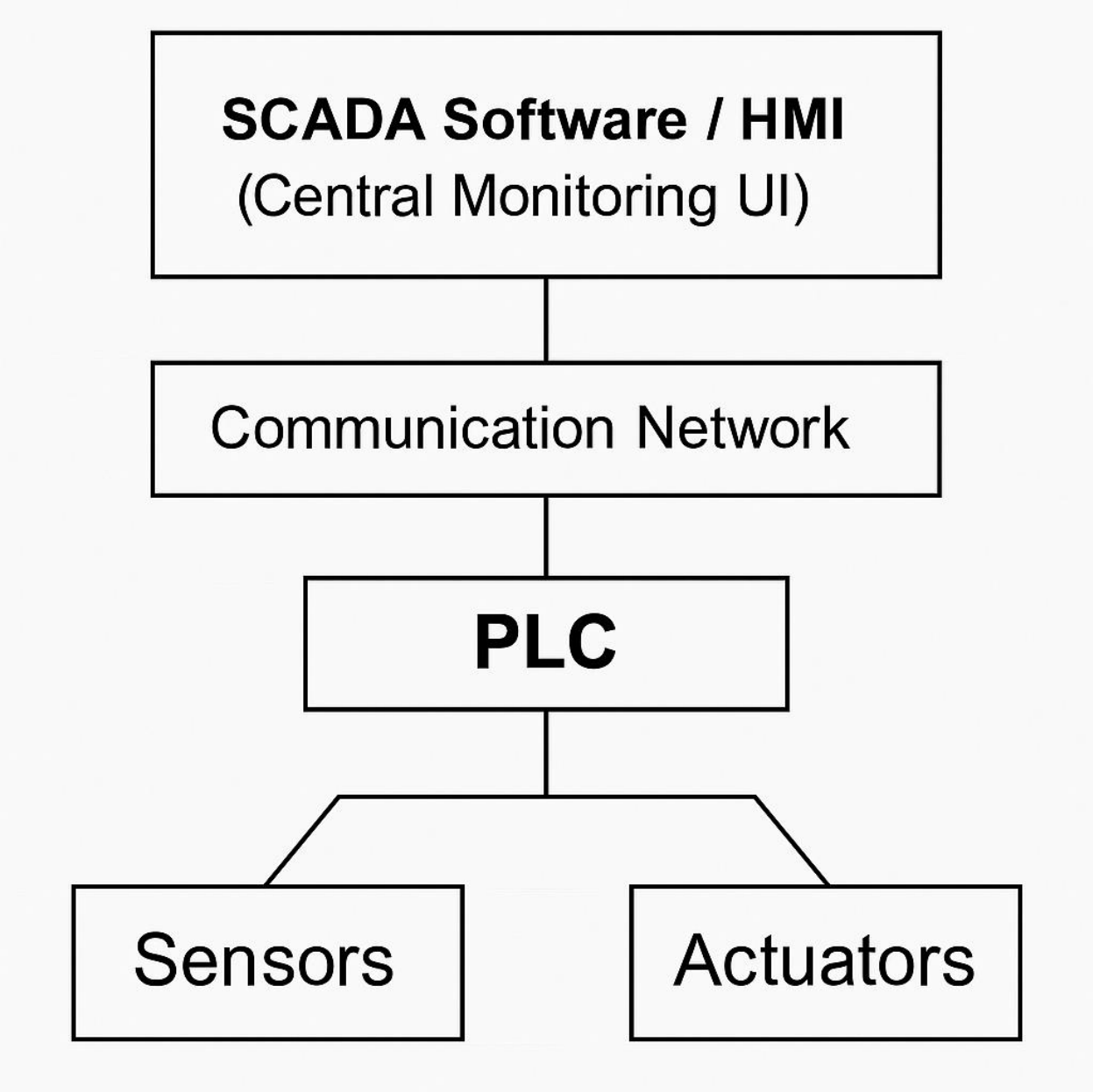
1. **HMI (Human-Machine Interface):**
    - The dashboard interface where operators monitor and control the process.
2. **SCADA Software:**
    - Runs on computers and processes data, logs, alarms, and controls.
3. **RTUs (Remote Terminal Units):**
    - Devices that collect sensor data and send it to the SCADA system.
4. **PLCs (Programmable Logic Controllers):**
    - Industrial computers used to automate electromechanical processes.

5. **Communication Infrastructure:**
    - Wired (Ethernet, RS485) or wireless (radio, cellular) to transfer data between RTUs/PLCs and the central system.

What SCADA Does

- **Monitor:** Real-time data from sensors (e.g., pressure, temperature).
- **Control:** Start/stop motors, open/close valves remotely.
- **Alarm:** Notify operators of abnormal conditions.
- **Data Logging:** Historical data storage for analytics, compliance, and maintenance.

SCADA Architecture:



SCADA vs IoT

Feature	SCADA	IoT
Architecture	Centralized	Decentralized/Cloud-based
Communication	Industrial protocols (MODBUS)	Internet protocols (MQTT, HTTP)
Scalability	Limited	High

Feature	SCADA	IoT
Intelligence	Operator-driven	Can be AI/ML-driven

Example Use Case: Water Treatment Plant (just for understanding)

- **Sensors**: pH, turbidity, flow rate
- **PLCs**: Control pumps, valves
- **RTUs**: Send data to SCADA
- **SCADA software**: Displays data, generates alarms, logs history
- **Operator**: Adjusts parameters based on dashboard insights

Nepal telecom also uses SCADA system (Siemens Spectrum Power 7).

Main RFID Protocol Standards

1. ISO/IEC 14443

- Used for contactless smart cards (e.g., NFC, passport chips)
- Operates at 13.56 MHz
- Communication range: < 10 cm
- Types A and B define different modulation methods and anti-collision

2. ISO/IEC 15693

- Also operates at 13.56 MHz
- Longer range (up to 1–1.5 meters)
- Used in library systems, asset tracking

3. ISO/IEC 18000

A family of standards defining air interface protocols for various frequency ranges:

- **18000-2**: LF
- **18000-3**: HF
- **18000-6**: UHF (widely used for long-range passive RFID)

4. EPCglobal Gen2 (ISO 18000-6C)

- The most popular protocol for UHF passive RFID
- Supports anti-collision and fast reading of multiple tags

- Read range: up to 12 meters
- Used in retail, logistics, supply chain

Unified Data Standards and Protocols

Introduction to Unified Data Standards

Unified data standards are agreed-upon specifications that enable consistent data representation, exchange, and interpretation across different systems, organizations, and industries. These standards are essential for interoperability, data integration, and efficient information sharing.

Importance of Data Standards

1. **Interoperability**: Enables different systems to communicate and exchange data
2. **Data Quality**: Ensures consistency, accuracy, and reliability of data
3. **Efficiency**: Reduces data conversion and transformation efforts
4. **Regulatory Compliance**: Helps meet legal and industry requirements
5. **Cost Reduction**: Minimizes data reconciliation and cleaning costs

Unified Data Standard defines:

- How data is formatted.
- How it is transmitted (protocol)
- How it is interpreted (sementic)

Key Protocols in IoT:

1. MQTT
2. CoAP
3. HTTP/HTTPs
4. AMQP
5. DOS
6. OPCVA
7. Zigbee/Z-wave

IEEE 802.15.4

- **IEEE 802.15.4** is a **wireless communication standard** developed by the **Institute of Electrical and Electronics Engineers (IEEE)**.
- It defines the **physical (PHY)** and **medium access control (MAC)** layers for low-rate wireless personal area networks (LR-WPANs).
- It focuses on low-power, low-cost communication for devices that require short range wireless networking, such as home automation and industrial automation,

Key Features:

Feature	Details
Data Rate	Low (20 kbps, 40 kbps, 250 kbps)
Range	Short-range (10 to 100 meters)
Power Consumption	Very low (good for battery-operated devices)
Cost	Low-cost implementation
Network Topologies	Star and Peer-to-Peer (Mesh possible with Zigbee)

Where is IEEE 802.15.4 Used?

IEEE 802.15.4 is the **foundation** for many IoT and wireless standards, such as:

- **Zigbee**
- **Thread**
- **6LoWPAN**
- **WirelessHART**

These protocols **build on top of** 802.15.4 to provide networking, security, and application support.

Layers Explained:

1. **Physical Layer (PHY):**
    - Handles modulation, demodulation, signal transmission.
    - Operates at **2.4 GHz**, **868 MHz**, or **915 MHz** bands.
2. **MAC Layer:**
    - Controls how devices **access the wireless medium**.
    - Handles **channel access**, **data framing**, **acknowledgements**, etc.

MODBUS Protocol

1. Introduction to Modbus

Modbus is a **serial communication protocol** developed by **Modicon (now Schneider Electric)** in **1979**. It is widely used in **industrial automation systems** for communication between **PLCs (Programmable Logic Controllers), sensors, and other industrial devices**.

Key Features:

- **Open standard** (freely available)
- **Simple and robust**
- **Supports multiple communication modes** (RTU, ASCII, TCP/IP)
- **Master-Slave architecture**

2. Modbus Communication Modes

Modbus supports three main communication modes:

Mode	Description	Usage
Modbus RTU	Binary format, compact & efficient	RS-485, RS-232
Modbus ASCII	Human-readable (hexadecimal)	Less common, slow
Modbus TCP/IP	Ethernet-based, uses TCP/IP	Industrial Ethernet

3. Modbus Architecture (Master-Slave)

- **Master (Client):** Initiates requests (e.g., PLC, SCADA, HMI).
- **Slave (Server):** Responds to requests (e.g., sensors, actuators).

4.How does it work?

- Modbus is transmitted over serial lines between devices. The simplest setup would be a single serial cable connecting the serial ports on two devices, a master and a slave.

- The data is sent as series of ones and zeroes called bits. Each bit is sent as a voltage. Zeroes are sent as positive voltages and ones as negative. The bits are sent very quickly. A typical transmission speed is 9600 baud (bits per second)

5.What is it used for?

- Modbus is typically used to transmit signals from instrumentation and control devices back to a main controller or data gathering system. For eg, a system that measures temperature and humidity and communicates the results to a computer.
- Modbus is often used to connect a supervisory computer with a remote terminal unit (RTU) in SCADA systems.

BACnet Protocol in Building Automation

BACnet (Building Automation and Control Networks) is a **communication protocol** designed for **building automation systems (BAS)**, such as HVAC, lighting, fire alarms, and access control. It enables different devices from various manufacturers to communicate seamlessly.

Overview of BACnet

- Developed by **ASHRAE (American Society of Heating, Refrigerating and Air-Conditioning Engineers)**.
- Standardized as **ISO 16484-5** and **ANSI/ASHRAE 135**.
- Used in **smart buildings** for energy efficiency and automation.

BACnet Architecture

BACnet follows a **layered model** similar to the OSI model but simplified:

BACnet Layer	Function
Physical Layer	Defines wired (Ethernet, RS-485) and wireless (BACnet/IP, Zigbee) connections.

BACnet Layer	Function
Network Layer	Handles addressing and routing (BACnet/IP, BACnet Ethernet, BACnet MS/TP).
Application Layer	Manages data exchange between devices (objects, services).

Key Components of BACnet

A. BACnet Devices

- Controllers (DDC - Direct Digital Controllers)
- Sensors (Temperature, Humidity, CO_2)
- Actuators (Valves, Dampers, Relays)
- Workstations (User Interface for Monitoring & Control)

B. BACnet Objects

Each device is represented as a collection of **standardized objects**:

- **Analog Input/Output** (e.g., temperature sensor)
- **Binary Input/Output** (e.g., light switch)
- **Device** (represents the whole device)
- **Schedule** (time-based automation)

C. BACnet Services

Used for communication between devices:

- **ReadProperty** – Reads data from an object.
- **WriteProperty** – Modifies an object's value.
- **Who-Is / I-Am** – Device discovery.
- **SubscribeCOV (Change of Value)** – Notifies when a value changes.

Advantages of BACnet

- **Interoperability** – Works with devices from different vendors.
- **Scalability** – Supports small to large buildings.
- **Flexibility** – Supports wired & wireless networks.
- **Open Standard** – No licensing fees.

Disadvantages of BACnet

- Limited Security in Older Versions (BACnet/IP without encryption).
- Slower Speeds in MS/TP compared to BACnet/IP.
- Complex Configuration for large networks.

Things to remember:

BACnet is the **leading protocol** in building automation, enabling **HVAC, lighting, and security systems** to communicate efficiently. While it has some security limitations, newer versions like **BACnet/SC** improve safety for modern smart buildings.

Zigbee

Intro:

- Zigbee is a low cost and low powered mesh network widely deployed for controlling and monitoring applications where it covers 10-100 meters within the range.
- Zigbee communication system is less expensive and simpler than the other proprietary short range wireless sensor networks as Bluetooth and wi-fi.
- Zigbee is a wireless communication protocol designed for low-power, low-data-rate, and short-range applications, commonly used in home automation, industrial control, and IoT (Internet of Things).
- It operates on the IEEE 802.15.4 standard and is known for its mesh networkin**g** capabilities, energy efficiency, and reliability.

Key Features of Zigbee:

1. **Low Power Consumption**
    - Ideal for battery-operated devices (e.g., sensors, smart lights).
    - Devices can sleep most of the time, waking only to transmit data.
2. **Mesh Networking**
    - Devices (nodes) can relay data, extending network coverage.
    - Self-healing: If one node fails, data reroutes through others.
3. **Short Range (10–100m per device)**
    - Range depends on environment and power output.
    - Mesh networking helps cover larger areas.
4. **Low Data Rate (250 kbps max)**
    - Suitable for small packets (e.g., sensor readings, on/off commands).
    - Not meant for high-bandwidth (like Wi-Fi).
5. **Secure Communication**

    - Supports **AES-128 encryption** for secure data transmission.
    - Uses **trust center** authentication in some networks.
6. **Multiple Topologies**
    - **Star, Tree, and Mesh** network configurations possible.
7. **Interoperability**
    - Devices from different brands can work together if they follow the same **Zigbee profile** (e.g., **Zigbee Home Automation (ZHA), Zigbee Light Link (ZLL)**).

Types of Zigbee Devices:

- Zigbee co-ordinator device:
    - communicates with zigbee router.
    - this device is used for connecting devices (1 per zigbee network)
- Zigbee router:
    - used for passing the data between devices.
- Zigbee end-device:
    - the device that is going to be controlled.

Zigbee Architecture

Zigbee follows a **layered architecture** based on the **IEEE 802.15.4** standard for low-rate wireless networks. It consists of **four main layers**, each handling specific functions to ensure reliable, low-power communication.

The Zigbee protocol stack is divided into:

A. Physical (PHY) Layer (IEEE 802.15.4)

- **Function:** Manages **radio transmission & reception**.
- **Frequency Bands:**
    - **2.4 GHz** (global, 16 channels, 250 kbps)
    - **915 MHz** (Americas, 10 channels, 40 kbps)
    - **868 MHz** (Europe, 1 channel, 20 kbps)
- **Modulation:** DSSS (Direct Sequence Spread Spectrum) for interference resistance.

B. MAC (Medium Access Control) Layer (IEEE 802.15.4)

- Function: Controls data framing, node addressing, and channel access.
- Supports:
    - CSMA/CA (Collision Avoidance) for efficient transmission.
    - Acknowledgment of received packets.
    - Beacon-enabled (scheduled) & non-beacon (unscheduled) modes.

C. Network (NWK) Layer (Zigbee-Specific)

- Function: Handles network formation, routing, and security.
- Key Tasks:
    - Network setup (coordinator initiates the network).
    - Device joining/leaving (authentication).
    - Mesh routing (AODV, tree routing).

D. Application (APL) Layer (Zigbee-Specific)

- Function: Defines how devices interact & communicate.
- It consists of application support sub-layer and zigbee device object (zdo).
- It contains manufacturer defined applications.

(just for knowledge)

Zigbee vs. Other Wireless Protocols

Feature	Zigbee	Wi-Fi	Bluetooth (BLE)	Z-Wave
Range	10–100m	50–100m	10–50m	30–100m
Data Rate	250 kbps	100+ Mbps	1–2 Mbps	100 kbps
Power Use	Very Low	High	Low	Very Low
Topology	Mesh	Star	Star/P2P	Mesh
Frequency	2.4 GHz, 915/868 MHz	2.4/5 GHz	2.4 GHz	800–900 MHz
Max Nodes	65,000+	Limited	7–8 active	232

Common Uses of Zigbee

- **Smart Home Devices** (lights, plugs, sensors, locks)
- **Industrial Automation** (sensors, monitoring)
- **Healthcare** (wearables, patient monitoring)
- **Retail & Logistics** (inventory tracking)

Zigbee Versions & Standards

- **Zigbee 3.0** (unifies earlier profiles for better compatibility)
- **Zigbee PRO** (enhanced for large-scale networks)
- **Zigbee Green Power** (for energy-harvesting devices)

Pros & Cons

✅ **Pros:**

✔Low power → long battery life

✔Scalable (supports thousands of devices)

✔Reliable mesh networking

❌ **Cons:**

✖Slower than Wi-Fi

✖Limited range per device (needs repeaters)

✖Some compatibility issues between brands

Popular Zigbee Devices & Hubs

- **Smart Hubs:** Samsung SmartThings, Amazon Echo (4th gen), Hubitat
- **Devices:** Philips Hue, Xiaomi Aqara, IKEA TRÅDFRI

NETWORK LAYER

Network Layer plays a crucial role in enabling communications between devices and ensuring data transmission from sensors/devices to cloud servers and other devices.

Functions of Network Layer

- Addressing and routing
    - assigns addresses IP/MAC to IoT devices.
    - routes data packets to the correct destination.
- Data transmission
    - transmits sensor data to server, apps and other devices.
- Protocol support
    - manages different network protocols (IPV6, MQTT, CoAP etc)
- Security and authentication
    - ensures safe communication.
- QoS
    - manages traffic prioritization and bandwidth usage.

CoAP (IETF Standard)

Overview of CoAP

CoAP (Constrained Application Protocol) is a lightweight, UDP-based communication protocol designed for resource-constrained IoT devices (e.g., sensors, actuators) operating in low-power, low-bandwidth networks. It is standardized in **RFC 7252** and is often compared to HTTP but optimized for IoT environments.

Purpose:

- Enable efficient communication between IoT devices and servers.
- Support machine-to-machine (M2M) interactions in constrained networks (e.g., LoRaWAN, 6LoWPAN).

Key Features

1. **Lightweight**:
    - Minimal header size (4 bytes vs. HTTP's 8+ bytes).
    - Designed for devices with limited RAM, CPU, or energy.
    - Uses UDP instead of TCP for faster transmission (no handshake overhead).
    - Optional reliability via **Confirmable (CON)** and **Non-Confirmable (NON)** messages.
2. **RESTful Architecture**:
    - Follows HTTP-like methods: GET, POST, PUT, DELETE.
    - Resources are identified by URIs (e.g., coap://device123/temperature).
3. **Observe Mode**:
    - Clients can "subscribe" to resources and receive updates automatically (similar to MQTT's publish-subscribe model).
4. **Security**:
    - Supports **DTLS (Datagram Transport Layer Security)** for encryption and authentication.

CoAP Architecture:

Application layer--------CoAP methods (GET,POST,PUT,DEL)

Application sub-layer (message layer)------handles CON,NON,ACK,RST

UDP Transport Layer--------UDP protocol

IP Layer----------IPV4/IPV6

Data Link & Physical----------IEE 802.15.4

- CoAP itself is an application layer protocol. So it only defines the application layer but relies on
    - UDP (Transport Layer)
    - IP (Network Layer)
    - IEEE 802.15.4 (MAC/PHY layers)

CoAP vs HTTP – Key Differences

Feature	CoAP (Constrained Application Protocol)	HTTP (Hypertext Transfer Protocol)
Purpose	Designed for **IoT** and constrained devices	Designed for **web apps**, browsers, and servers
Transport Protocol	**UDP** (lightweight, connectionless)	**TCP** (reliable, connection-oriented)
Message Format	Binary, very compact	Text-based (ASCII), larger
Header Size	~4 bytes	~100+ bytes
Security	DTLS (for UDP)	TLS (for TCP/HTTPS)
URI Format	coap://192.168.1.10/temp	http://example.com/temp

CoAP vs. MQTT

Feature	CoAP	MQTT
Transport	UDP	TCP
Messaging	Request-response + Observe	Publish-subscribe
Overhead	Minimal	Moderate
Use Case	Constrained devices, real-time	Broker-based IoT ecosystems

CoAP in IoT Applications

- **Smart Homes**: Lightweight communication between sensors (e.g., motion detectors) and controllers.
- **Industrial IoT (IIoT)**: Monitoring machinery health with low-power sensors.
- **Environmental Monitoring**: Transmitting data from remote weather stations.
- **Healthcare**: Wearable devices sending patient vitals to a central server.

Example Use Case

Smart Agriculture:

1. Soil moisture sensors (CoAP clients) send `POST` requests to a CoAP server.
2. The server processes data and triggers irrigation actuators via `PUT` commands.
3. Farmers use a CoAP-enabled app to `GET` real-time soil status.

6LoWPAN

6LoWPAN stands for **IPv6 over Low-power Wireless Personal Area Networks**.
It is a **networking protocol** that allows **IPv6 packets** to be sent and received over **low-power, low-data-rate wireless networks** such as those based on **IEEE 802.15.4** (used in Zigbee, for example).

Why is 6LoWPAN used in IoT?

IoT devices like sensors and smart appliances are usually:

- Small and battery-powered
- Use low bandwidth
- Need to communicate over short distances

6LoWPAN helps these devices connect to the Internet using IPv6, enabling seamless communication and integration with other devices and systems.

Key Features of 6LoWPAN

1. **IPv6 Support**
   Allows devices to use **Internet Protocol version 6**, which supports a huge number of unique IP addresses.
2. **Header Compression**
   IPv6 headers are large (40 bytes), which is too much for small packets in IoT. 6LoWPAN compresses them to fit into tiny frames.
3. **Mesh Networking**
   Devices can forward data to other nodes (multi-hop), extending range and reliability.
4. **Low Power Consumption**
   Designed for energy-efficient operation, perfect for battery-powered sensors

How It Works

- Devices use **IEEE 802.15.4** (low-power wireless) to communicate.
- 6LoWPAN **adapts IPv6 packets** so they can be transmitted over this wireless network.
- At the destination, the compressed IPv6 packets are **reconstructed** back to standard form.

Architecture:

Application Layer----------where IoT app runs

Transport Layer----------UDP(used due to simplicity and low overhead)

Network Layer----------IPV6 internet protocol

Adaptation Layer----------6LoWPAN (compression, fragmentation)

(note: adaptation layer ->sub layer of Network here)

Data Link Layer----------IEEE 802.15.4 (MAC layer)

Physical----------Radio Hardware (2.4 GHz)

Example Use in IoT

- A **smart agriculture system** where soil sensors (using 6LoWPAN) send data like temperature and moisture to a central server over the internet.
- Home automation devices (lights, doors) using 6LoWPAN to communicate securely and efficiently.

Security Requirements in IoT

The **Internet of Things (IoT)** connects various devices to the internet, enabling data exchange and automation. However, this connectivity also introduces security risks. Below are the key **security requirements** in IoT systems:

1. Device Authentication

- Ensures that only authorized devices can connect to the IoT network.
- Methods: **Passwords, Digital Certificates, Biometrics, Two-Factor Authentication (2FA).**

2. Data Encryption

- Protects data from unauthorized access during transmission and storage.
- Common protocols: **TLS/SSL, AES (Advanced Encryption Standard).**

3. Secure Communication Protocols

- Ensures safe data transfer between IoT devices and servers.
- Examples: **MQTT with TLS, HTTPS, CoAP with DTLS.**

4. Access Control & Authorization

- Restricts user/device access based on roles (e.g., admin, user, guest).

5. Firmware & Software Security

- Regular updates to patch vulnerabilities.
- Secure boot mechanisms to prevent unauthorized firmware changes.

6. Physical Security

- Protects IoT devices from tampering or theft.
- Methods: **Tamper-proof casing.**

7. Network Security

- Protects the IoT network from attacks like **DDoS, Man-in-the-Middle (MITM).**
- Solutions: **Firewalls, Intrusion Detection Systems (IDS), Network Segmentation.**

8. Privacy Protection

- Ensures user data is collected and stored responsibly.
- Complies with regulations like **GDPR (General Data Protection Regulation).**

9. Secure APIs

- APIs used for IoT communication must be protected against attacks.
- Methods: **API Gateways, OAuth 2.0, Rate Limiting.**

10. Logging & Monitoring

- Tracks device activities to detect and respond to security threats.
